

Học sâu và ứng dụng Generative Adversarial Networks (GAN)

Giới thiệu về GAN

- Mô hình GAN được giới thiệu bởi Ian J. Goodfellow vào năm 2014 và đã đạt được rất nhiều thành công lớn trong Deep Learning nói riêng và AI nói chung.
- Mô tả về GAN: "The most interesting idea in the last 10 years in Machine Learning".
- GAN: Mô hình mạng học sâu tạo sinh được ứng dụng để sinh ra nội dung mới dựa trên nội dung đã học được

Ứng dụng của GAN

- Generate Photographs of Human Faces
 - Ví dụ về ảnh mặt người do GAN sinh ra từ 2014 đến 2017. Mọi người có thể thấy chất lượng ảnh sinh ra tốt lên đáng kể theo thời gian.



2014



2015



2016



2017

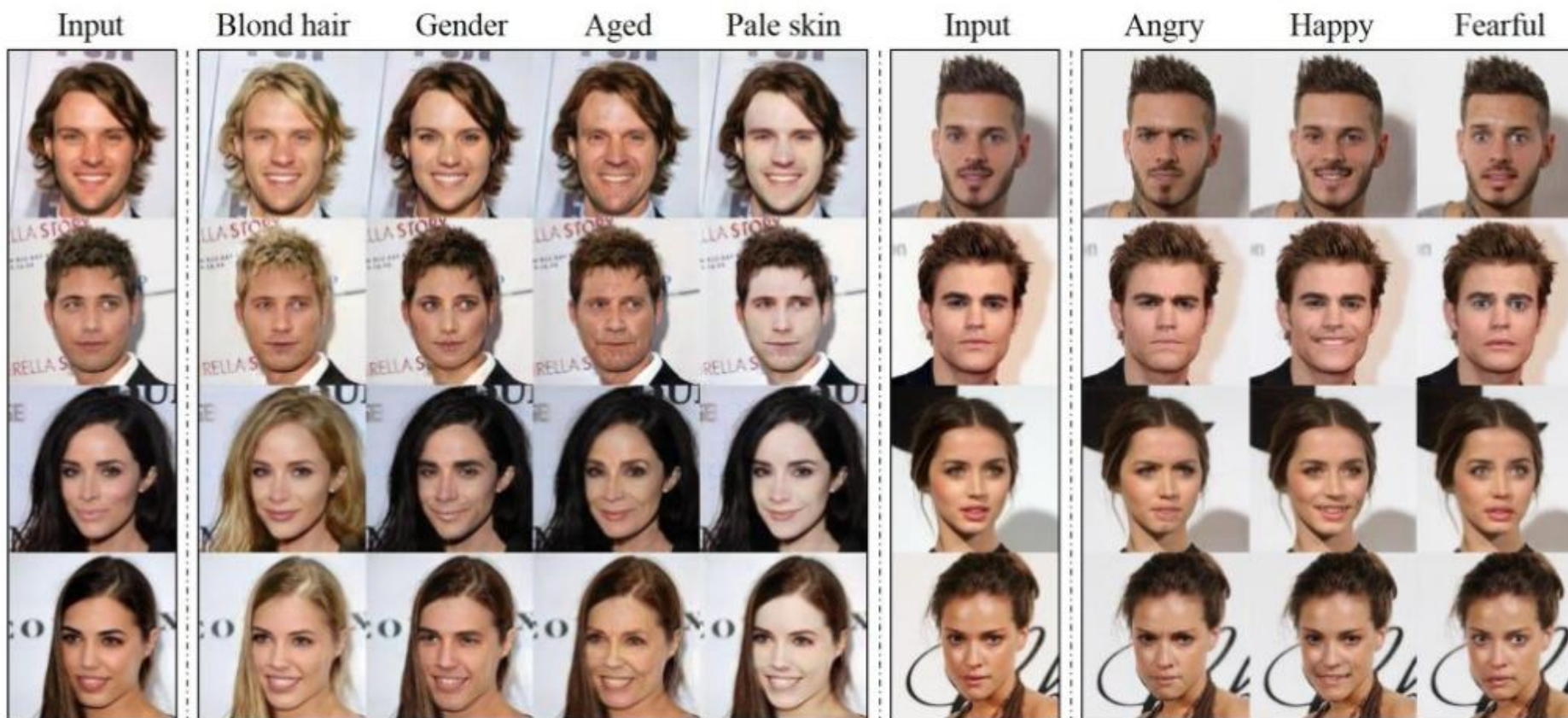
Ứng dụng của GAN

- Generate Photographs of Human Faces (StyleGAN)
 - ảnh sinh ra bởi GAN năm 2018, phải để ý rất chi tiết thì mới có thể phân biệt được ảnh mặt đây là sinh ra hay ảnh thật.



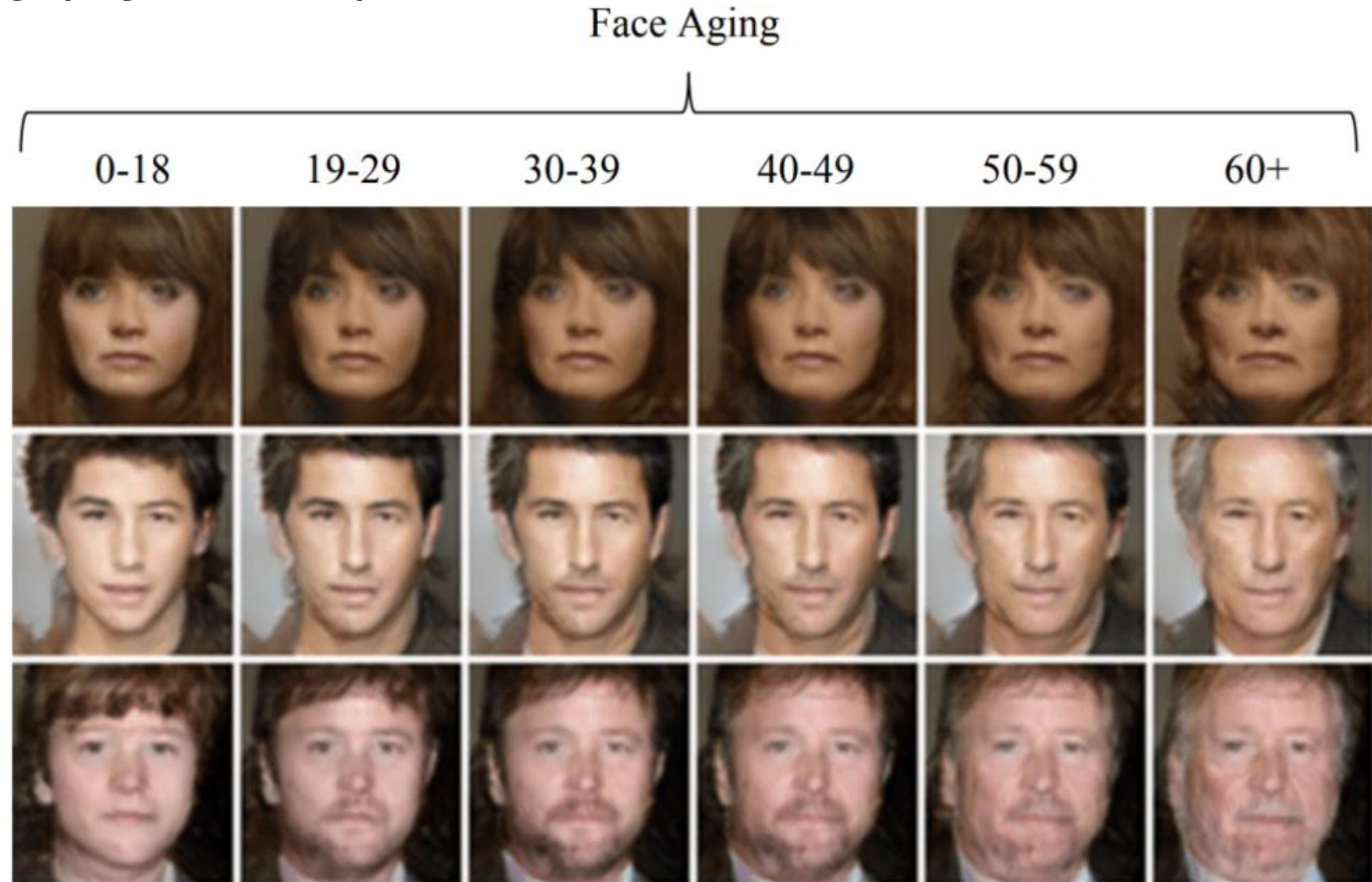
Ứng dụng của GAN

- Image editing (StarGAN)
 - FaceApp làm mưa làm gió trong thời gian vừa qua.
 - Đó là một ứng dụng của GAN để sửa các thuộc tính của khuôn mặt như màu tóc, da, giới tính, cảm xúc hay độ tuổi.



Ứng dụng của GAN

- Image editing (Age-cGAN)



Ứng dụng của GAN

- Generate Anime characters



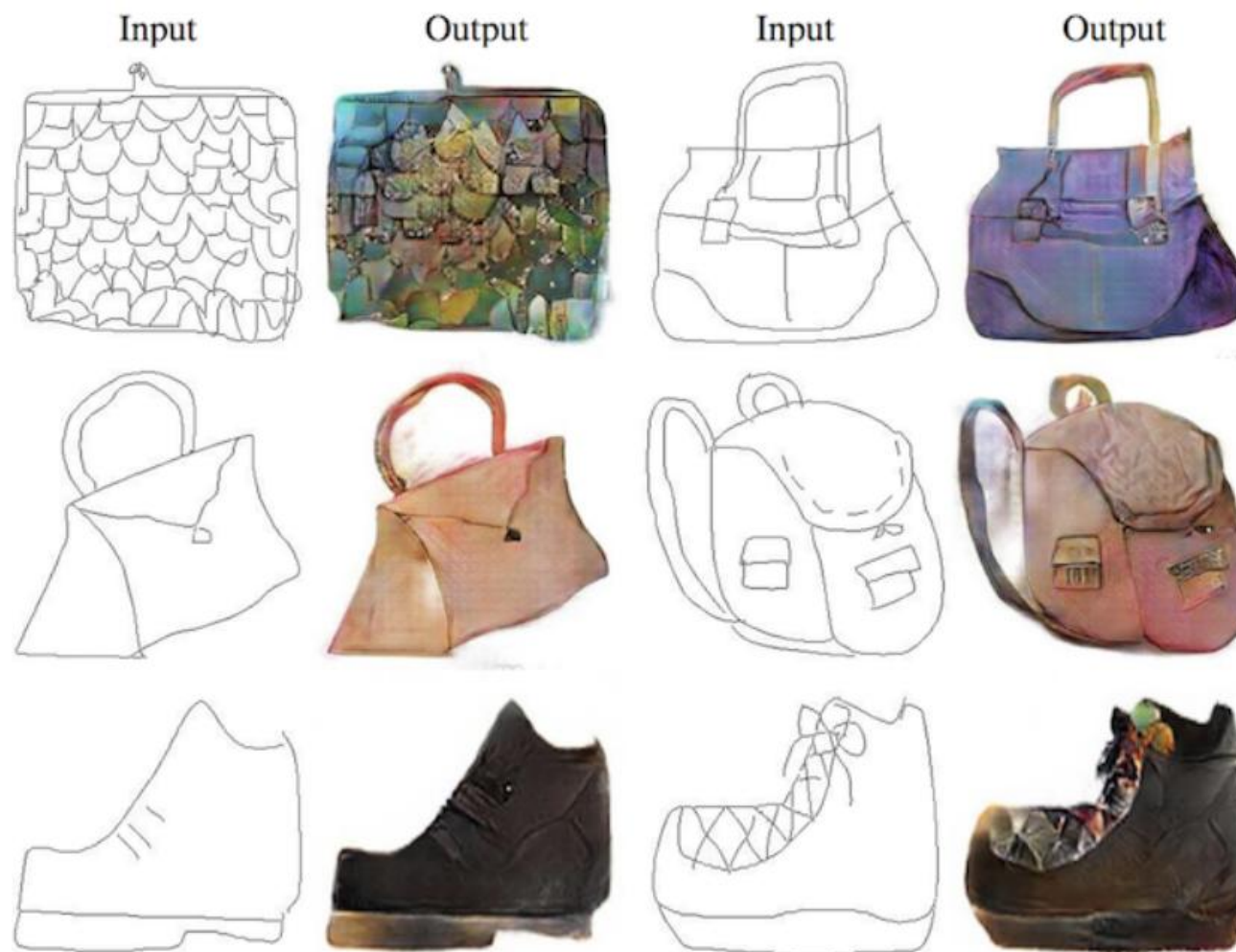
Ứng dụng của GAN

- Generate Realistic Photographs (BigGAN)
 - Năm 2018, Andrew Brock cho ra paper bigGAN với có khả năng sinh ra các ảnh tự nhiên rất khó phân biệt với ảnh chụp thường.



Ứng dụng của GAN

- Image-to-Image Translation (Pix2pix)
 - Input là 1 ảnh và output là 1 ảnh tương ứng, ví dụ input là ảnh không màu, output là ảnh màu.



Ứng dụng của GAN

- Unsupervised Image-to-image translation
 - Bài toán trên (Image-to-image translation) là supervised tức là ta có các dữ liệu thành cặp (input, output) như bản phác thảo của cái cặp và ảnh màu của nó.
 - Khi muốn chuyển dữ liệu từ domain này sang domain khác (từ ngựa thường sang ngựa vằn) thì ta không có sẵn các cặp dữ liệu để huấn luyện, đó gọi là bài toán unsupervised.

Ứng dụng của GAN

Zebras ↔ Horses



zebra → horse



horse → zebra

Ứng dụng của GAN

- Super-resolution (SRGAN)
 - GAN có thể dùng để tăng chất lượng của ảnh từ độ phân giải thấp lên độ phân giải cao với kết quả rất tốt.



Ứng dụng của GAN

- Text to image (StackGAN)
 - GAN có thể học sinh ra ảnh với input là một câu.



Ứng dụng của GAN

- Generate new human pose
 - GAN có thể sinh ra người với dáng đứng mới.



Ứng dụng của GAN

- Music generation (MidiNet)
 - GAN có thể áp dụng để sinh ra những bản nhạc.



(a) MidiNet model 1



(b) MidiNet model 2



(c) MidiNet model 3

GAN là gì?

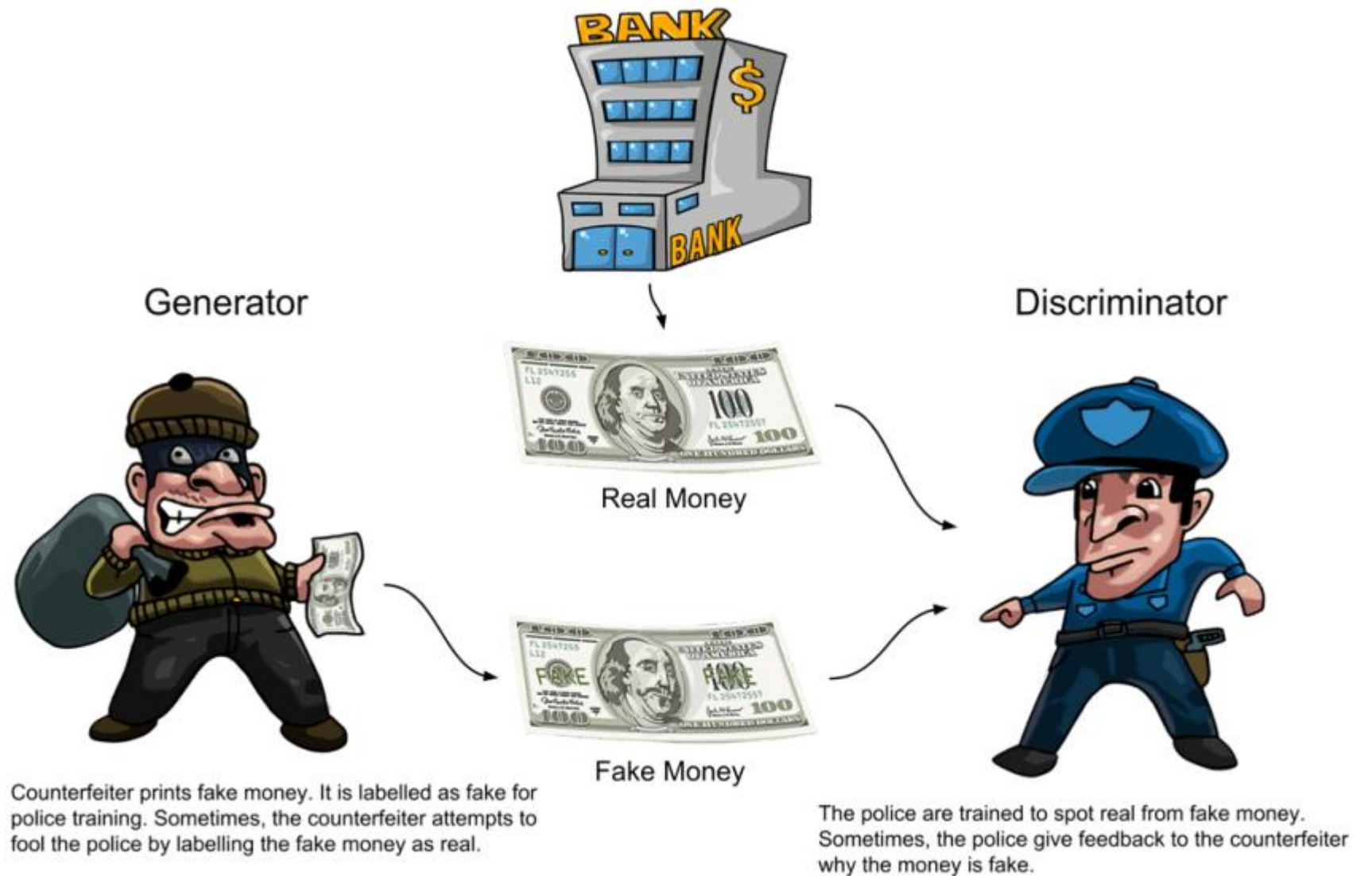
- GAN thuộc nhóm generative model
 - Generative là tính từ nghĩa là khả năng sinh ra.
 - model nghĩa là mô hình
 - generative model nghĩa là mô hình có khả năng sinh ra dữ liệu hay GAN là mô hình có khả năng sinh ra dữ liệu mới.
- GAN viết tắt cho Generative Adversarial Networks
 - Generative giống như ở trên, Network có nghĩa là mạng (mô hình), còn Adversarial là đối nghịch.
 - GAN được cấu thành từ 2 mạng gọi là Generator và Discriminator, luôn đối nghịch đầu với nhau trong quá trình huấn luyện mạng GAN.

Cấu trúc mạng GAN

- GAN cấu tạo gồm 2 mạng là Generator và Discriminator
 - Generator sinh ra các dữ liệu giống như thật
 - Discriminator cố gắng phân biệt đâu là dữ liệu được sinh ra từ Generator và đâu là dữ liệu thật có.

Cấu trúc mạng GAN

- Bài toán giờ là dùng GAN để generate ra tiền giả mà có thể dùng để chi tiêu được.
- Dữ liệu có là tiền thật.



Cấu trúc mạng GAN

- GAN sinh tiền giả
 - Generator giống như người làm tiền giả còn Discriminator giống như cảnh sát.
 - Người làm tiền giả sẽ cố gắng làm ra tiền giả mà cảnh sát cũng không phân biệt được.
 - Còn cảnh sát sẽ phân biệt đâu là tiền thật và đâu là tiền giả.
 - Mục tiêu cuối cùng là người làm tiền giả sẽ làm ra tiền mà cảnh sát cũng không phân biệt được đâu là thật và đâu là giả và có thể mang tiền đi tiêu được.

Cấu trúc mạng GAN

- Trong quá trình huấn luyện GAN thì cảnh sát có 2 việc:
 - Học cách phân biệt tiền nào là thật, tiền nào là giả.
 - Nói cho thằng làm tiền giả biết là tiền nó làm ra vẫn chưa qua mắt được và cần cải thiện hơn.
 - Dần dần thì thằng làm tiền giả sẽ làm tiền giống tiền thật hơn và cảnh sát cũng thành thạo việc phân biệt tiền giả và tiền thật.
 - Mong đợi là tiền giả từ GAN sẽ đánh lừa được cảnh sát.

Cấu trúc mạng GAN

- Ý tưởng của GAN bắt nguồn từ zero-sum non-cooperative game
 - Như trò chơi đối kháng 2 người (cờ vua, cờ tướng), nếu một người thắng thì người còn lại sẽ thua.
 - Ở mỗi lượt thì cả hai đều muốn maximize cơ hội thắng của tôi và minimize cơ hội thắng của đối phương.
 - Discriminator và Generator trong mạng GAN giống như 2 đối thủ trong trò chơi.
 - Trong lý thuyết trò chơi thì GAN model hội tụ (converge) khi cả Generator và Discriminator đạt tới trạng thái Nash equilibrium, tức là 2 người chơi đạt trạng thái cân bằng và đi tiếp các bước không làm tăng cơ hội thắng.

Cấu trúc mạng GAN

- **Bài toán:** Dùng mạng GAN sinh ra các chữ số viết tay giống với dữ liệu trong [MNIST dataset](#).

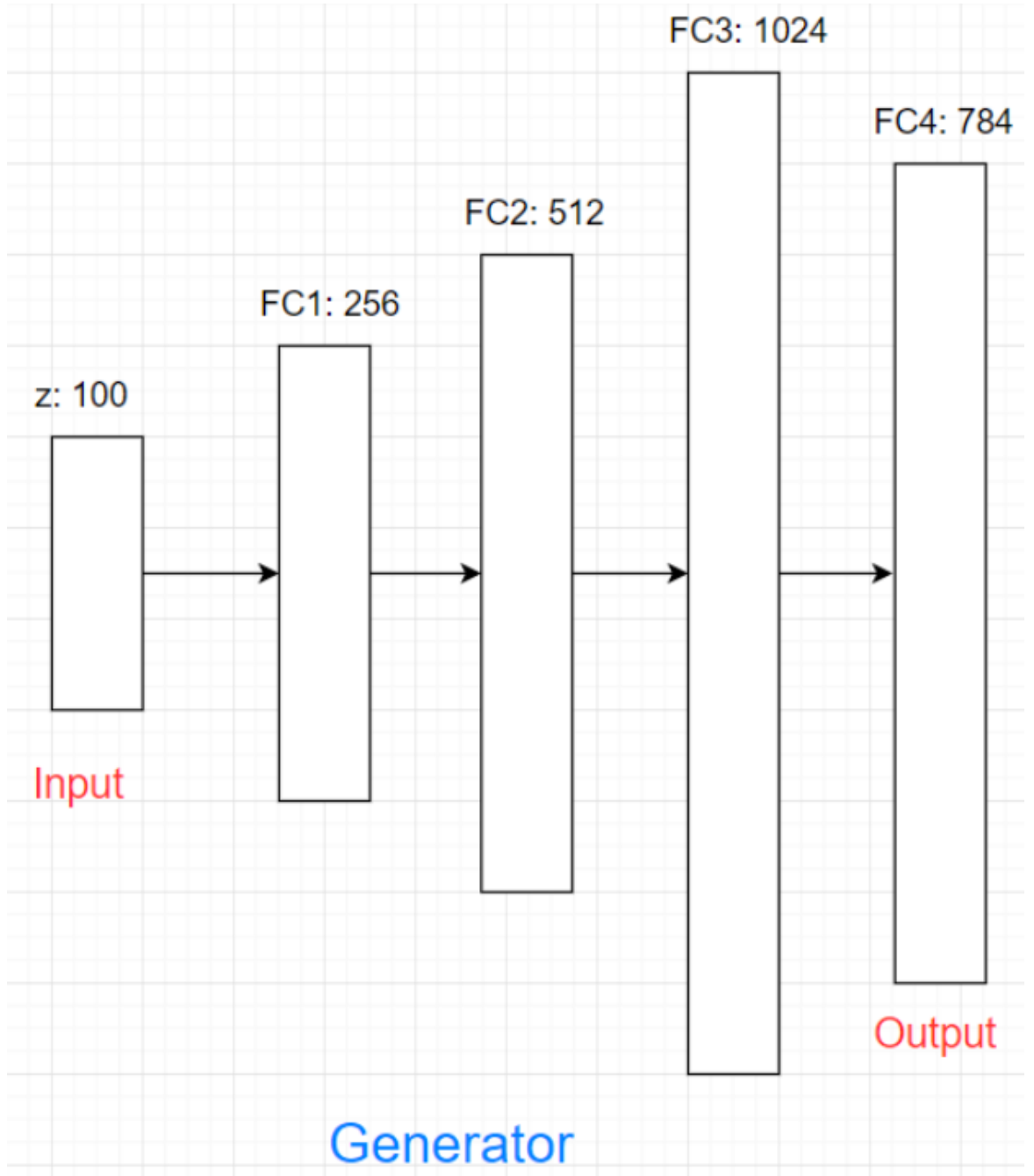


Cấu trúc mạng GAN

- Generator
 - Sinh ra các chữ số giống với dữ liệu trong MNIST dataset
 - Generator có input là noise (random vector) là output là chữ số.
 - Tại sao input là noise?
 - Vì các chữ số khi viết ra không hoàn toàn giống nhau.
 - Ví dụ số 0 ở hàng đầu tiên có rất nhiều biến dạng nhưng vẫn là số 0.
 - input của Generator là noise để khi ta thay đổi noise ngẫu nhiên thì Generator có thể sinh ra một biến dạng khác của chữ viết tay.
 - Noise cho Generator thường được sinh ra từ normal distribution hoặc uniform distribution.

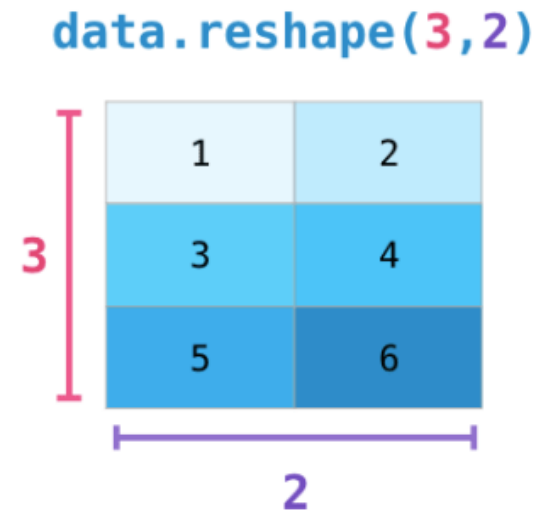
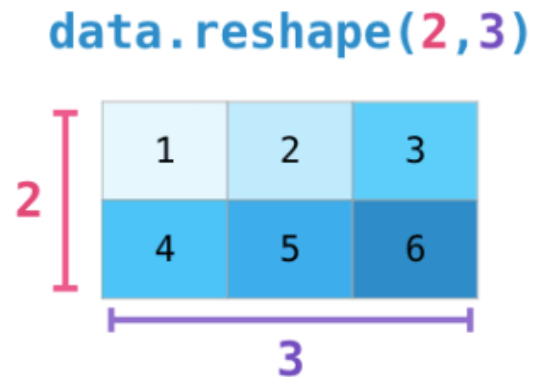
Cấu trúc mạng GAN

- Generator
 - Input của Generator là noise vector 100 chiều.
 - Mô hình neural network được áp dụng với số nút trong hidden layer lần lượt là 256, 512, 1024.
 - Output layer có số chiều là 784 vì output đầu ra là ảnh giống với dữ liệu MNIST, ảnh xám kích thước 28×28 (784 pixel).



Cấu trúc mạng GAN

- Generator
 - Output là vector kích thước 784×1 sẽ được reshape về 28×28 đúng định dạng của dữ liệu MNIST.



Cấu trúc mạng GAN

- Generator

Mô hình Generator

```
g = Sequential()  
g.add(Dense(256, input_dim=z_dim,  
activation=LeakyReLU(alpha=0.2)))  
g.add(Dense(512, activation=LeakyReLU(alpha=0.2)))  
g.add(Dense(1024, activation=LeakyReLU(alpha=0.2)))
```

Vì dữ liệu ảnh MNIST được chuẩn hóa về [0, 1] nên hàm G khi sinh ảnh ra cũng cần sinh ra ảnh có pixel value trong khoảng [0, 1] => hàm sigmoid được chọn

```
g.add(Dense(784, activation='sigmoid'))  
g.compile(loss='binary_crossentropy', optimizer=adam,  
metrics=['accuracy'])
```

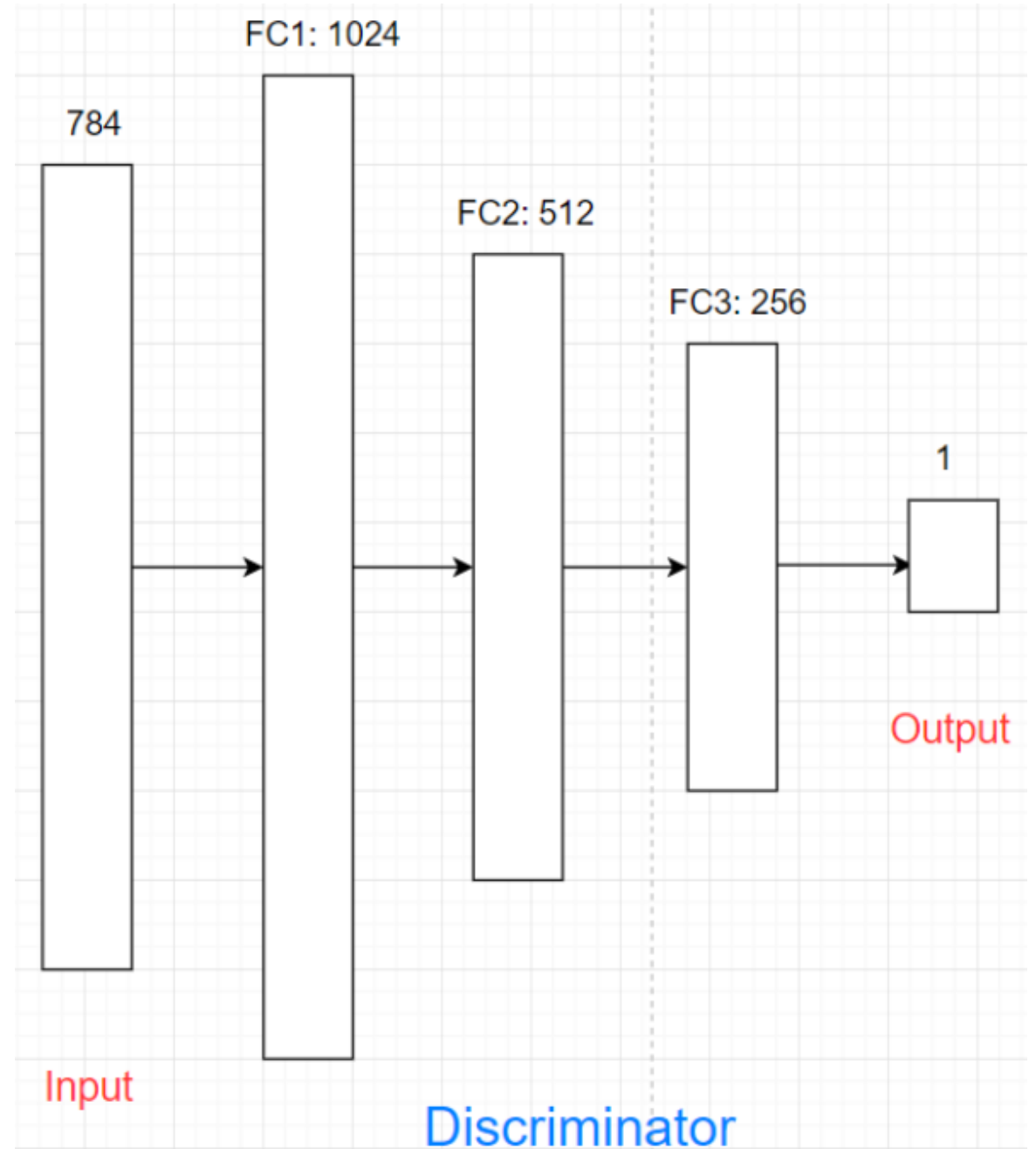
Cấu trúc mạng GAN

- Discriminator
 - Discriminator dùng để phân biệt chữ số từ bộ MNIST và dữ liệu được sinh ra từ Generator.
 - Discriminator có input là ảnh biểu diễn bằng 784 chiều, output là ảnh thật hay ảnh giả.
 - Đây là bài toán binary classification, giống với logistic regression.

Cấu trúc mạng GAN

- Discriminator

- Input của Discriminator là ảnh kích thước 784 chiều.
- Sau đây mô hình neural network được áp dụng với số node trong hidden layer lần lượt là 1024, 512, 256. Mô hình đối xứng lại với Generator.
- Output là 1 node thể hiện xác suất ảnh input là ảnh thật, hàm sigmoid được sử dụng.



Cấu trúc mạng GAN

- Discriminator

```
# Mô hình Discriminator
d = Sequential()
d.add(Dense(1024, input_dim=784, activation=LeakyReLU(alpha=0.2)))
d.add(Dropout(0.3))
d.add(Dense(512, activation=LeakyReLU(alpha=0.2)))
d.add(Dropout(0.3))
d.add(Dense(256, activation=LeakyReLU(alpha=0.2)))
d.add(Dropout(0.3))
# Hàm sigmoid cho bài toán binary classification
d.add(Dense(1, activation='sigmoid'))
```

Cấu trúc mạng GAN

- Loss function
 - Kí hiệu z là noise đầu vào của generator, x là dữ liệu thật từ bộ dataset.
 - Kí hiệu mạng Generator là G , mạng Discriminator là D . $G(z)$ là ảnh được sinh ra từ Generator. $D(x)$ là giá trị dự đoán của Discriminator xem ảnh x là thật hay không, $D(G(z))$ là giá trị dự đoán xem ảnh sinh ra từ Generator là ảnh thật hay không.
 - Cần thiết kế hai loss function cho hai mạng Generator và Discriminator.

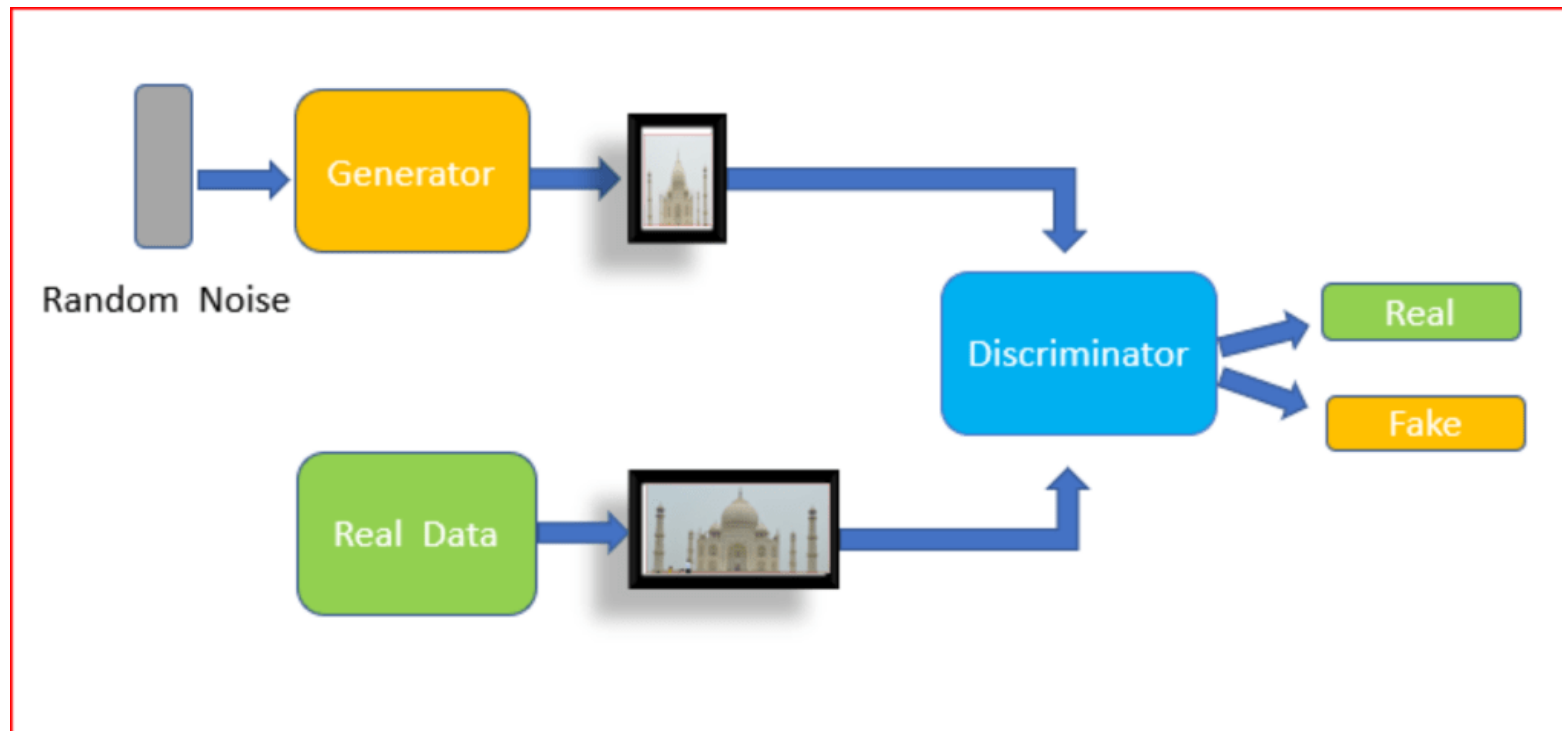
Cấu trúc mạng GAN

- Loss function
 - Discriminator thì cố gắng phân biệt đâu là ảnh thật và đâu là ảnh giả nên là bài toán binary classification nên loss function dùng giống với binary cross-entropy.

```
d.compile(loss='binary_crossentropy', optimizer=adam, metrics=['accuracy'])
```

Cấu trúc mạng GAN

- Loss function
 - Giá trị output của model qua hàm sigmoid nên sẽ trong (0, 1) nên Discriminator sẽ được huấn luyện để input ảnh ở dataset thì output gần 1, còn input là ảnh được sinh ra từ Generator thì output gần 0, hay $D(x) \rightarrow 1$ còn $D(G(z)) \rightarrow 0$.



Cấu trúc mạng GAN

- Loss function
 - loss function muốn maximize $D(x)$ và minimize $D(G(z))$.
 - minimize $D(G(z))$ tương đương với maximize $(1 - D(G(z)))$, do đó loss function của Discriminator như sau:

$$\max_D V(D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

recognize real images better

recognize generated images better

- E là kì vọng, hiểu đơn giản là lấy trung bình của tất cả dữ liệu, hay maximize $D(x)$ với x là dữ liệu trong training set.

Cấu trúc mạng GAN

- Loss function

- Generator sẽ học để đánh lừa Discriminator rằng số nó sinh ra là số thật hay $D(G(z)) \rightarrow 1$. Hay loss function muốn maximize $D(G(z))$, tương đương với minimize $(1 - D(G(z)))$

$$\min_G V(G) = \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Optimize G that can fool the discriminator the most.

- Do đó ta có thể viết gộp lại loss của mô hình GAN:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

Cấu trúc mạng GAN

- Loss function
 - Từ hàm loss của GAN có thể thấy rằng việc huấn luyện Generator và Discriminator đối nghịch nhau
 - D cố gắng maximize loss.
 - G cố gắng minimize loss.
 - Quá trình huấn luyện GAN kết thúc khi model GAN đạt đến trạng thái cân bằng của 2 models, gọi là Nash equilibrium.

Cấu trúc mạng GAN

- Code
 - Khi huấn luyện Discriminator, ta lấy BATCH_SIZE (128) ảnh thật từ dataset với label là 1
 - Sinh thêm 128 noise vector (sinh ra từ normal distribution) sau đó cho noise vector qua Generator để tạo ảnh fake, ảnh fake này có label là 0.
 - Sau đó 128 ảnh thật + 128 ảnh fake được dùng để train Discriminator.

Cấu trúc mạng GAN

- Code

```
# Lấy ngẫu nhiên các ảnh từ MNIST dataset (ảnh thật)
image_batch = X_train[np.random.randint(0, X_train.shape[0], size=BATCH_SIZE)]

# Sinh ra noise ngẫu nhiên
noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))

# Dùng Generator sinh ra ảnh từ noise
generated_images = g.predict(noise)

X = np.concatenate((image_batch, generated_images))

# Tạo label
y = np.zeros(2*BATCH_SIZE)

y[:BATCH_SIZE] = 0.9 # gán label bằng 1 cho những ảnh từ MNIST dataset và 0 cho ảnh
được sinh ra bởi Generator

# Train discriminator
d.trainable = True

d_loss = d.train_on_batch(X, y)
```

Cấu trúc mạng GAN

- Code

- Khi huấn luyện Generator ta cũng sinh ra 128 ảnh fake từ noise vector, nhưng ta gán label cho ảnh giả là 1 vì ta muốn nó học để đánh lừa được Discriminator. Không cập nhật các hệ số của Discriminator vì mỗi mạng có mục tiêu riêng.

```
# Train generator
noise = np.random.normal(0, 1, size=(BATCH_SIZE, z_dim))
# Khi train Generator gán label bằng 1 cho những ảnh sinh ra bởi Generator -> cố
găng lừa Discriminator.
y2 = np.ones(BATCH_SIZE)
# Khi train Generator thì không cập nhật hệ số của Discriminator.
d.trainable = False
g_loss = gan.train_on_batch(noise, y2)
```